Python Video Splitting Code

https://gemini.google.com/app/6cf0552946efeec5

User prompt: Make me a Python code that cuts a video file into equal 30-second-long clips and renames the files in numerical order

Response: You can split a video into 30-second numbered clips in Python using either **MoviePy** (easiest setup, frame-exact cuts) or **FFmpeg via `subprocess`** (near-instant, no quality loss).

Option 1: Using MoviePy (Frame-Accurate & Easy Setup)

Install MoviePy first:

```
pip install moviepy
```

This script loads your video, slices it into 30-second chunks, and names the output files sequentially (`001.mp4`, `002.mp4`, etc.) inside an output folder.

```
import os
import math
from moviepy import VideoFileClip  # Note: MoviePy v2.0+ syntax

def split_video_into_clips(input_video_path: str, output_folder: str, clip_length: int = 30, keep_remainder: bool =
    os.makedirs(output_folder, exist_ok=True)

    with VideoFileClip(input_video_path) as video:
        total_duration = int(video.duration)

        # Calculate number of clips
        num_clips = math.ceil(total_duration / clip_length) if keep_remainder else (total_duration // clip_length)

        for i in range(num_clips):
            start_time = i * clip_length
            end_time = min((i + 1) * clip_length, total_duration)

            # Create subclip (use .subclip() instead of .subclipped() if on MoviePy 1.x)
            subclip = video.subclipped(start_time, end_time)

            # Format filename in numerical order: 001.mp4, 002.mp4, etc.
            output_filename = f"{i + 1:03d}.mp4"
            output_path = os.path.join(output_folder, output_filename)

            print(f"Exporting {output_filename} ({start_time}s to {end_time}s)...")
            subclip.write_videofile(
                output_path,
                codec="libx264",
                audio_codec="aac",
                temp_audiofile="temp-audio.m4a",
                remove_temp=True,
                logger=None  # Set to "bar" if you want a progress bar
            )

    print(f"Done! Created {num_clips} clips in '{output_folder}'.")

if __name__ == "__main__":
    split_video_into_clips(
        input_video_path="input_video.mp4",
        output_folder="output_clips",
        clip_length=30,
        keep_remainder=False  # Set to True to keep the final partial clip if under 30s
    )
```

- **Zero-padding (`{i + 1:03d}.mp4`):** Pads numbers with leading zeros (`001.mp4`, `002.mp4`) so files stay in strict numerical order in your file explorer. Change it to `{i + 1}.mp4` if you prefer `1.mp4`, `2.mp4`.
- **Remainder handling:** By default, `keep_remainder=False` drops any leftover tail shorter than 30 seconds so every clip is strictly 30 seconds long. Set `keep_remainder=True` to include the leftover segment at the end.

Option 2: Using FFmpeg via `subprocess` (10x Faster, No Re-encoding)

Because MoviePy re-encodes every frame, long videos can take several minutes. If you have **FFmpeg** installed on your system, calling its built-in segmenter from Python splits a video in a few seconds by copying the original streams directly:

```
import os
import subprocess

def fast_split_video(input_video_path: str, output_folder: str, clip_length: int = 30):
    os.makedirs(output_folder, exist_ok=True)

    # %03d automatically numbers files starting from 001.mp4, 002.mp4...
    output_pattern = os.path.join(output_folder, "%03d.mp4")
```

```
    command = [
        "ffmpeg",
        "-i", input_video_path,
        "-c", "copy",                  # Copy video/audio without re-encoding
        "-map", "0",
        "-segment_time", str(clip_length),
        "-f", "segment",
        "-reset_timestamps", "1",      # Ensure each clip's timer starts at 00:00
        "-segment_start_number", "1",  # Start numbering at 1 instead of 0
        output_pattern
    ]

    subprocess.run(command, check=True)
    print(f"Done! Clips saved to '{output_folder}'.")

if __name__ == "__main__":
    fast_split_video("input_video.mp4", "output_clips", clip_length=30)
```

User prompt: Update the MoviePy script to also crop each 30-second clip into a 9:16 vertical aspect ratio for TikTok, Reels, and YouTube Shorts.

Response: This updated script calculates a centered **9:16 vertical crop** from your source video, rounds the dimensions to even numbers (required by the `libx264` MP4 encoder), and slices the video into numbered 30-second clips.

Updated MoviePy Script (9:16 Center Crop)

```
import os
import math
from moviepy import VideoFileClip  # MoviePy v2.0+

def split_and_crop_vertical(
    input_video_path: str,
    output_folder: str,
    clip_length: int = 30,
    keep_remainder: bool = False,
    resize_to_1080p: bool = False
):
    os.makedirs(output_folder, exist_ok=True)

    with VideoFileClip(input_video_path) as video:
        total_duration = int(video.duration)
        w, h = video.size
        target_ratio = 9 / 16

        # Calculate 9:16 crop dimensions from the center
        if (w / h) > target_ratio:
            # Standard landscape video (e.g., 16:9) -> crop left and right sides
            new_h = h
            new_w = int(h * target_ratio)
        else:
            # Taller than 9:16 -> crop top and bottom
            new_w = w
            new_h = int(w / target_ratio)

        # H.264 (libx264) requires width and height to be divisible by 2
        new_w -= new_w % 2
        new_h -= new_h % 2

        # Center crop coordinates
        x1 = (w - new_w) // 2
        y1 = (h - new_h) // 2

        # Apply the crop once to the full video stream before slicing
        # (Use .crop() instead of .cropped() if on MoviePy 1.x)
        cropped_video = video.cropped(x1=x1, y1=y1, width=new_w, height=new_h)

        # Optional: Upscale to standard 1080x1920 resolution
        if resize_to_1080p:
            # (Use .resize() instead of .resized() if on MoviePy 1.x)
            cropped_video = cropped_video.resized((1080, 1920))

        num_clips = (
            math.ceil(total_duration / clip_length)
            if keep_remainder
            else (total_duration // clip_length)
        )
```

```
        for i in range(num_clips):
            start_time = i * clip_length
            end_time = min((i + 1) * clip_length, total_duration)

            # Slice the already-cropped video (use .subclip() on MoviePy 1.x)
            subclip = cropped_video.subclipped(start_time, end_time)

            output_filename = f"{i + 1:03d}.mp4"
            output_path = os.path.join(output_folder, output_filename)

            print(f"Exporting {output_filename} ({start_time}s to {end_time}s) at {cropped_video.size[0]}x{cropped_v
            subclip.write_videofile(
                output_path,
                codec="libx264",
                audio_codec="aac",
                temp_audiofile="temp-audio.m4a",
                remove_temp=True,
                logger=None
            )

    print(f"Done! Created {num_clips} vertical 9:16 clips in '{output_folder}'.")

if __name__ == "__main__":
    split_and_crop_vertical(
        input_video_path="input_video.mp4",
        output_folder="shorts_clips",
        clip_length=30,
        keep_remainder=False,
        resize_to_1080p=False  # Set True to force 1080x1920 output
    )
```

Key Details in This Update

- **Even-Pixel Rounding (`new_w -= new_w % 2`):** Cropping a standard `1920x1080` video to 9:16 yields `607.5x1080`. Because the `libx264` encoder crashes if a dimension is an odd number, the script rounds down to `606x1080`.
- **Cropping Before Slicing:** Calling `video.cropped(...)` once before the `for` loop applies a lazy transformation to the parent video stream, keeping the loop clean and fast.
- **`resize_to_1080p` Toggle:** TikTok, Reels, and YouTube Shorts accept `606x1080` directly and render it faster. If you need strict `1080x1920` files, set `resize_to_1080p=True`.
- **MoviePy 1.x Compatibility:** If you are on an older version of MoviePy (below v2.0), replace `.cropped()`, `.resized()`, and `.subclipped()` with `.crop()`, `.resize()`, and `.subclip()`.

User prompt: Update the MoviePy script so that instead of cutting off the sides, it places the full 16:9 video centered over a blurred 9:16 background.

Response: To place the full 16:9 frame inside a 9:16 vertical canvas (`1080x1920`), this updated script builds two layers from your video using MoviePy's `CompositeVideoClip`:

1. **Background layer:** Zooms in to fill the full `1920` pixel height, crops the sides to `1080` pixels wide, removes the duplicate audio track, and applies a Gaussian blur using `Pillow` (which is already installed with MoviePy).
2. **Foreground layer:** Scales the original 16:9 video down to `1080` pixels wide and centers it vertically on top of the blurred background.

Updated MoviePy Script (Blurred 9:16 Background + Centered 16:9 Video)

```
import os
import math
import numpy as np
from PIL import Image, ImageFilter
from moviepy import VideoFileClip, CompositeVideoClip  # MoviePy v2.0+

def make_blur_filter(blur_radius: int = 30):
    """Returns a frame filter function that applies a Gaussian blur."""
    def blur_frame(frame):
        img = Image.fromarray(frame)
        blurred_img = img.filter(ImageFilter.GaussianBlur(radius=blur_radius))
        return np.array(blurred_img)
    return blur_frame

def split_with_blurred_background(
    input_video_path: str,
    output_folder: str,
    clip_length: int = 30,
    keep_remainder: bool = False,
    blur_radius: int = 30,
```

```
    target_resolution: tuple = (1080, 1920)
):
    os.makedirs(output_folder, exist_ok=True)
    target_w, target_h = target_resolution

    with VideoFileClip(input_video_path) as video:
        total_duration = int(video.duration)

        # --- 1. Build the 9:16 Blurred Background Layer ---
        # Scale height to 1920 (or width to 1080 if source is narrower than 9:16)
        bg = video.without_audio().resized(height=target_h)
        if bg.w < target_w:
            bg = bg.resized(width=target_w)

        # Center-crop the zoomed background to 1080x1920
        x1 = (bg.w - target_w) // 2
        y1 = (bg.h - target_h) // 2
        bg = bg.cropped(x1=x1, y1=y1, width=target_w, height=target_h)

        # Blur each frame of the background
        bg = bg.image_transform(make_blur_filter(blur_radius))

        # --- 2. Build the Centered 16:9 Foreground Layer ---
        fg = video.resized(width=target_w).with_position("center")

        # --- 3. Stack Foreground Over Background ---
        composite_video = CompositeVideoClip([bg, fg], size=(target_w, target_h))

        # --- 4. Slice into Numbered 30-Second Clips ---
        num_clips = (
            math.ceil(total_duration / clip_length)
            if keep_remainder
            else (total_duration // clip_length)
        )

        for i in range(num_clips):
            start_time = i * clip_length
            end_time = min((i + 1) * clip_length, total_duration)

            subclip = composite_video.subclipped(start_time, end_time)

            output_filename = f"{i + 1:03d}.mp4"
            output_path = os.path.join(output_folder, output_filename)

            print(f"Exporting {output_filename} ({start_time}s to {end_time}s) at {target_w}x{target_h}...")
            subclip.write_videofile(
                output_path,
                codec="libx264",
                audio_codec="aac",
                temp_audiofile="temp-audio.m4a",
                remove_temp=True,
                logger=None
            )

    print(f"Done! Created {num_clips} blurred-background clips in '{output_folder}'.")

if __name__ == "__main__":
    split_with_blurred_background(
        input_video_path="input_video.mp4",
        output_folder="shorts_blurred_clips",
        clip_length=30,
        keep_remainder=False,
        blur_radius=30  # Increase for a heavier blur, decrease for a softer blur
    )
```

Key Details in This Update

- **Preventing Double Audio (`video.without_audio()`):** `CompositeVideoClip` automatically mixes the audio tracks of every clip inside it. Stripping the audio from the background layer prevents the final video from playing two overlapping audio tracks at double volume.
- **No Extra Libraries Needed:** MoviePy already installs `numpy` and `Pillow` (`PIL`) under the hood, so `ImageFilter.GaussianBlur` works out of the box.
- **Slightly Zoomed Foreground Option:** If the 16:9 video feels too small on a phone screen, you can zoom the foreground slightly so it fills more of the vertical space—for example, change `video.resized(width=target_w)` to `video.resized(width=int(target_w * 1.25))` and `CompositeVideoClip` will automatically trim the excess edges.
- **MoviePy 1.x Compatibility:** If you are on MoviePy 1.x, replace `.resized()`, `.cropped()`, `.image_transform()`, `.with_position()`, and `.subclipped()` with `.resize()`, `.crop()`, `.fl_image()`, `.set_position()`, and `.subclip()`.

User prompt: Show me how to create the exact same blurred 9:16 background effect and 30-second split using FFmpeg via subprocess for faster rendering.

Response: By using FFmpeg's `filter_complex` and built-in `segment` muxer inside a single command, you can generate the blurred 9:16 background, overlay the centered 16:9 video, and slice the output into numbered 30-second clips in **one continuous pass**—without passing frames back and forth through Python.

Single-Pass FFmpeg Script (Blurred 9:16 + 30s Split)

```
import os
import math
import subprocess

def get_video_duration(input_video_path: str) -> float:
    """Uses ffprobe (included with FFmpeg) to get the exact video duration in seconds."""
    command = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format=duration",
        "-of", "default=noprint_wrappers=1:nokey=1",
        input_video_path
    ]
    result = subprocess.run(command, capture_output=True, text=True, check=True)
    return float(result.stdout.strip())

def ffmpeg_blurred_vertical_split(
    input_video_path: str,
    output_folder: str,
    clip_length: int = 30,
    keep_remainder: bool = False,
    blur_sigma: int = 35,
    preset: str = "fast",
    crf: int = 20
):
    os.makedirs(output_folder, exist_ok=True)
    output_pattern = os.path.join(output_folder, "%03d.mp4")

    # 1. Build the FFmpeg filtergraph:
    # - Split video stream into [bg] and [fg]
    # - [bg]: Fill 1080x1920, center-crop, and apply Gaussian blur
    # - [fg]: Scale width to 1080 (keeping aspect ratio & even height)
    # - Overlay [fg] dead-center on [bg]
    filter_complex = (
        f"[0:v]split=2[bg][fg];"
        f"[bg]scale=1080:1920:force_original_aspect_ratio=increase,"
        f"crop=1080:1920,gblur=sigma={blur_sigma}[bg_blurred];"
        f"[fg]scale=1080:-2[fg_scaled];"
        f"[bg_blurred][fg_scaled]overlay=(W-w)/2:(H-h)/2,format=yuv420p[outv]"
    )

    command = [
        "ffmpeg",
        "-y",
        "-i", input_video_path,
    ]

    # 2. If dropping the remainder, stop encoding at the last full 30s multiple
    if not keep_remainder:
        total_duration = get_video_duration(input_video_path)
        usable_duration = (int(total_duration) // clip_length) * clip_length
        if usable_duration == 0:
            raise ValueError(f"Video is shorter than {clip_length} seconds.")
        command.extend(["-t", str(usable_duration)])

    # 3. Apply filtergraph, encoding settings, and exact-keyframe segmenting
    command.extend([
        "-filter_complex", filter_complex,
        "-map", "[outv]",
        "-map", "0:a?",  # Map audio if present ('?' prevents crash on silent videos)
        "-c:v", "libx264",
        "-preset", preset,
        "-crf", str(crf),
        "-c:a", "aac",
        "-b:a", "192k",
        # Force a keyframe at every multiple of clip_length so cuts are frame-exact
        "-force_key_frames", f"expr:gte(t,n_forced*{clip_length})",
        "-f", "segment",
        "-segment_time", str(clip_length),
```

```
        "-segment_start_number", "1",
        "-reset_timestamps", "1",
        output_pattern
    ])

    print("Running FFmpeg single-pass render and split...")
    subprocess.run(command, check=True)
    print(f"Done! Clips saved to '{output_folder}'.")

if __name__ == "__main__":
    ffmpeg_blurred_vertical_split(
        input_video_path="input_video.mp4",
        output_folder="shorts_ffmpeg_clips",
        clip_length=30,
        keep_remainder=False,
        blur_sigma=35  # Higher = heavier background blur
    )
```

How This Works Under the Hood

- **Single-Pass Filtergraph (`-filter_complex`):** Instead of opening and re-encoding each 30-second slice separately, FFmpeg clones the video stream in memory (`split=2[bg][fg]`), blurs the background with C-optimized Gaussian blur (`gblur`), overlays the sharp 16:9 foreground in the center (`overlay=(W-w)/2:(H-h)/2`), and feeds the combined stream directly to the segmenter.
- **Frame-Exact 30-Second Cuts (`-force_key_frames`):** FFmpeg's segmenter can only cut video on keyframes (I-frames). Passing `expr:gte(t,n_forced*30)` forces the encoder to place a keyframe at `30s`, `60s`, `90s`, and so on, ensuring every clip is strictly 30 seconds long.
- **Zero Wasted Encoding on Remainder (`-t`):** When `keep_remainder=False`, `ffprobe` checks the total length first and tells FFmpeg to stop encoding as soon as the final full 30-second clip finishes, saving processing time on the leftover tail.
- **Even-Dimension Protection (`scale=1080:-2`):** The `-2` height flag automatically calculates the proportional height of your foreground video and rounds it to the nearest even pixel so `libx264` never fails.

User prompt: How do I modify this FFmpeg Python script to use GPU hardware acceleration like NVIDIA NVENC or Apple VideoToolbox for even faster rendering?

Response: Switching from `libx264` (CPU) to **NVIDIA NVENC** (`h264_nvenc`) or **Apple VideoToolbox** (`h264_videotoolbox`) requires three changes to your FFmpeg pipeline:

1. **Hardware Decoding (`-hwaccel auto`):** Placed before `-i` so your GPU decodes the incoming video stream instead of your CPU.
2. **GPU Encoder Flags:** Replaces `-c:v libx264 -crf 20` with hardware-specific quality controls (`-cq` for NVIDIA or `-q:v` for Apple Silicon).
3. **Fast Downscale-Blur Trick (`filter_complex`):** Calculating a heavy Gaussian blur on a full `1080x1920` frame on the CPU will bottleneck your GPU. Scaling the background down to `270x480` first, blurring it with a smaller radius, and scaling it back up to `1080x1920` produces an identical heavy blur while running **over 10x faster**.

GPU-Accelerated FFmpeg Script (NVIDIA, Apple Silicon, AMD & Intel)

```
import os
import platform
import subprocess

def get_video_duration(input_video_path: str) -> float:
    """Uses ffprobe to get the exact video duration in seconds."""
    command = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format=duration",
        "-of", "default=noprint_wrappers=1:nokey=1",
        input_video_path
    ]
    result = subprocess.run(command, capture_output=True, text=True, check=True)
    return float(result.stdout.strip())

def get_gpu_encoder_args(hardware: str = "auto", quality: int = 20) -> list:
    """
    Returns the appropriate FFmpeg video encoder flags for the selected GPU.
    hardware options: 'auto', 'nvidia', 'apple', 'intel', 'amd', 'cpu'
    """
    if hardware == "auto":
        # Check macOS first for Apple VideoToolbox
        if platform.system() == "Darwin":
            hardware = "apple"
        else:
            # Query FFmpeg for available hardware encoders
```

```
            res = subprocess.run(
                ["ffmpeg", "-hide_banner", "-encoders"],
                capture_output=True, text=True
            )
            encoders_out = res.stdout
            if "h264_nvenc" in encoders_out:
                hardware = "nvidia"
            elif "h264_qsv" in encoders_out:
                hardware = "intel"
            elif "h264_amf" in encoders_out:
                hardware = "amd"
            else:
                hardware = "cpu"

        print(f"Using encoder profile: {hardware.upper()}")

        if hardware == "nvidia":
            # NVIDIA NVENC: p4 = medium/fast preset, -cq = constant quality (0-51, lower is better)
            return [
                "-c:v", "h264_nvenc",
                "-preset", "p4",
                "-rc", "vbr",
                "-cq", str(quality),
                "-b:v", "0"
            ]
        elif hardware == "apple":
            # Apple VideoToolbox: -q:v scale is 1 to 100 (higher is better; 65 is ~CRF 20)
            apple_q = max(1, min(100, 100 - int(quality * 1.75)))
            return [
                "-c:v", "h264_videotoolbox",
                "-q:v", str(apple_q),
                "-allow_sw", "1"  # Fallback to software if hardware is busy
            ]
        elif hardware == "intel":
            # Intel QuickSync (QSV)
            return [
                "-c:v", "h264_qsv",
                "-preset", "fast",
                "-global_quality", str(quality)
            ]
        elif hardware == "amd":
            # AMD AMF
            return [
                "-c:v", "h264_amf",
                "-quality", "balanced",
                "-rc", "cqp",
                "-qp_i", str(quality),
                "-qp_p", str(quality)
            ]
        else:
            # Standard CPU fallback
            return [
                "-c:v", "libx264",
                "-preset", "fast",
                "-crf", str(quality)
            ]

    def ffmpeg_gpu_blurred_vertical_split(
        input_video_path: str,
        output_folder: str,
        clip_length: int = 30,
        keep_remainder: bool = False,
        hardware: str = "auto",
        quality: int = 20
    ):
        os.makedirs(output_folder, exist_ok=True)
        output_pattern = os.path.join(output_folder, "%03d.mp4")

        # Fast Blur Optimization:
        # Downscale background by 4x (270x480) -> apply sigma=10 blur -> upscale to 1080x1920.
        # Visually identical to sigma=40 at 1080p, but prevents CPU filter bottlenecks.
        filter_complex = (
            "[0:v]split=2[bg][fg];"
            "[bg]scale=270:480:force_original_aspect_ratio=increase,"
            "crop=270:480,gblur=sigma=10,scale=1080:1920:flags=bicubic[bg_blurred];"
            "[fg]scale=1080:-2[fg_scaled];"
            "[bg_blurred][fg_scaled]overlay=(W-w)/2:(H-h)/2,format=yuv420p[outv]"
        )
```

```
    command = [
        "ffmpeg",
        "-y",
        "-hwaccel", "auto",  # Decode input video on GPU when supported
        "-i", input_video_path,
    ]

    if not keep_remainder:
        total_duration = get_video_duration(input_video_path)
        usable_duration = (int(total_duration) // clip_length) * clip_length
        if usable_duration == 0:
            raise ValueError(f"Video is shorter than {clip_length} seconds.")
        command.extend(["-t", str(usable_duration)])

    # Combine filtergraph, GPU encoder settings, and segmenter
    command.extend([
        "-filter_complex", filter_complex,
        "-map", "[outv]",
        "-map", "0:a?",
        *get_gpu_encoder_args(hardware=hardware, quality=quality),
        "-c:a", "aac",
        "-b:a", "192k",
        "-force_key_frames", f"expr:gte(t,n_forced*{clip_length})",
        "-f", "segment",
        "-segment_time", str(clip_length),
        "-segment_start_number", "1",
        "-reset_timestamps", "1",
        output_pattern
    ])

    print("Running GPU-accelerated FFmpeg render and split...")
    subprocess.run(command, check=True)
    print(f"Done! Clips saved to '{output_folder}'.")

if __name__ == "__main__":
    ffmpeg_gpu_blurred_vertical_split(
        input_video_path="input_video.mp4",
        output_folder="shorts_gpu_clips",
        clip_length=30,
        keep_remainder=False,
        hardware="auto",  # Set to "nvidia", "apple", "intel", "amd", or "auto"
        quality=20        # Lower = higher quality (20 is visually near-lossless)
    )
```

GPU Encoder Flag Reference

Because hardware encoders do not use `libx264`'s `-crf` flag, here is how each GPU controls quality and speed in the script above:

Platform	FFmpeg Encoder	Quality Flag Used	Speed Preset Flag	Notes
NVIDIA (NVENC)	`h264_nvenc`	`-rc vbr -cq 20 -b:v 0`	`-preset p1` (fastest) to `p7` (best)	`p4` balances speed and compression; requires NVIDIA driver 450+
Apple Silicon (M1–M4)	`h264_videotoolbox`	`-q:v 65` (1–100 scale, higher is better)	Handled automatically by macOS Media Engine	On older Intel Macs where `-q:v` isn't supported, replace `-q:v 65` with `-b:v 8M`
Intel QuickSync	`h264_qsv`	`-global_quality 20`	`-preset veryfast` to `veryslow`	Works on Intel CPUs with integrated graphics or Intel Arc GPUs
AMD Radeon (AMF)	`h264_amf`	`-rc cqp -qp_i 20 -qp_p 20`	`-quality speed` or `balanced`	Uses Constant Quantization Parameter mode

---

User prompt: progress bar

Response: To animate a progress bar inside a single-pass FFmpeg command, there is one classic FFmpeg quirk to work around: the `drawbox` filter only calculates width once at startup and cannot animate frame-by-frame.

Instead, the fastest and most reliable technique is to generate a solid color bar and slide it across a semi-transparent background track using the `overlay` filter's frame-by-frame timestamp variable (`t`):

- **Auto-resetting every 30 seconds:** Using the formula `t/30 - floor(t/30)` calculates the exact progress (`0.0` to `1.0`) of each 30-second segment and automatically resets the bar to `0%` at `30s`, `60s`, `90s`, etc.—without needing commas that break FFmpeg's filter parser.
- **Smart placement (`video_bottom`):** By default, this script attaches the progress bar directly to the **bottom edge of the 16:9 video** rather than the very bottom of the 9:16 screen, where TikTok, Reels, and Shorts cover the frame with captions and account names.

GPU-Accelerated FFmpeg Script with Animated Progress Bar

```
import os
import platform
import subprocess

def get_video_duration(input_video_path: str) -> float:
    """Uses ffprobe to get the exact video duration in seconds."""
    command = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format=duration",
        "-of", "default=noprint_wrappers=1:nokey=1",
        input_video_path
    ]
    result = subprocess.run(command, capture_output=True, text=True, check=True)
    return float(result.stdout.strip())

def get_gpu_encoder_args(hardware: str = "auto", quality: int = 20) -> list:
    """Returns FFmpeg video encoder flags for NVIDIA, Apple Silicon, Intel, AMD, or CPU."""
    if hardware == "auto":
        if platform.system() == "Darwin":
            hardware = "apple"
        else:
            res = subprocess.run(
                ["ffmpeg", "-hide_banner", "-encoders"],
                capture_output=True, text=True
            )
            encoders_out = res.stdout
            if "h264_nvenc" in encoders_out:
                hardware = "nvidia"
            elif "h264_qsv" in encoders_out:
                hardware = "intel"
            elif "h264_amf" in encoders_out:
                hardware = "amd"
            else:
                hardware = "cpu"

    print(f"Using encoder profile: {hardware.upper()}")

    if hardware == "nvidia":
        return ["-c:v", "h264_nvenc", "-preset", "p4", "-rc", "vbr", "-cq", str(quality), "-b:v", "0"]
    elif hardware == "apple":
        apple_q = max(1, min(100, 100 - int(quality * 1.75)))
        return ["-c:v", "h264_videotoolbox", "-q:v", str(apple_q), "-allow_sw", "1"]
    elif hardware == "intel":
        return ["-c:v", "h264_qsv", "-preset", "fast", "-global_quality", str(quality)]
    elif hardware == "amd":
        return ["-c:v", "h264_amf", "-quality", "balanced", "-rc", "cqp", "-qp_i", str(quality), "-qp_p", str(qualit
    else:
        return ["-c:v", "libx264", "-preset", "fast", "-crf", str(quality)]

def ffmpeg_gpu_split_with_progress_bar(
    input_video_path: str,
    output_folder: str,
    clip_length: int = 30,
    keep_remainder: bool = False,
    hardware: str = "auto",
    quality: int = 20,
    bar_height: int = 12,
    bar_color: str = "#FF0050",        # Hex color or name (e.g., '#FF0050', '#00F2FE', 'white')
    bar_position: str = "video_bottom" # 'video_bottom', 'screen_top', or 'screen_bottom'
):
    os.makedirs(output_folder, exist_ok=True)
    output_pattern = os.path.join(output_folder, "%03d.mp4")

    # Expression that goes from -W (0% visible) to 0 (100% visible) every `clip_length` seconds
    progress_x_expr = f"-W+W*(t/{clip_length}-floor(t/{clip_length}))"

    # Build the filtergraph based on where the progress bar should sit
    if bar_position == "video_bottom":
        # Attaches the progress bar flush to the bottom edge of the centered 16:9 video
        filter_complex = (
            f"[0:v]split=3[bg][fg][bar_src];"
            f"[bg]scale=270:480:force_original_aspect_ratio=increase,"
            f"crop=270:480,gblur=sigma=10,scale=1080:1920:flags=bicubic[bg_blurred];"
            f"[fg]scale=1080:-2,"
            f"drawbox=x=0:y=ih-{bar_height}:w=iw:h={bar_height}:color=black@0.45:t=fill[fg_track];"
            f"[bar_src]scale=1080:{bar_height},drawbox=color={bar_color}:t=fill[bar];"
```

```
            f"[fg_track][bar]overlay=x='{progress_x_expr}':y=H-h[fg_with_bar];"
            f"[bg_blurred][fg_with_bar]overlay=(W-w)/2:(H-h)/2,format=yuv420p[outv]"
        )
    else:
        # Places the progress bar at the very top or very bottom of the 9:16 canvas
        y_pos = "0" if bar_position == "screen_top" else f"1920-{bar_height}"
        filter_complex = (
            f"[0:v]split=3[bg][fg][bar_src];"
            f"[bg]scale=270:480:force_original_aspect_ratio=increase,"
            f"crop=270:480,gblur=sigma=10,scale=1080:1920:flags=bicubic[bg_blurred];"
            f"[fg]scale=1080:-2[fg_scaled];"
            f"[bg_blurred][fg_scaled]overlay=(W-w)/2:(H-h)/2,"
            f"drawbox=x=0:y={y_pos}:w=1080:h={bar_height}:color=black@0.45:t=fill[canvas_track];"
            f"[bar_src]scale=1080:{bar_height},drawbox=color={bar_color}:t=fill[bar];"
            f"[canvas_track][bar]overlay=x='{progress_x_expr}':y={y_pos},format=yuv420p[outv]"
        )

    command = [
        "ffmpeg",
        "-y",
        "-hwaccel", "auto",
        "-i", input_video_path,
    ]

    if not keep_remainder:
        total_duration = get_video_duration(input_video_path)
        usable_duration = (int(total_duration) // clip_length) * clip_length
        if usable_duration == 0:
            raise ValueError(f"Video is shorter than {clip_length} seconds.")
        command.extend(["-t", str(usable_duration)])

    command.extend([
        "-filter_complex", filter_complex,
        "-map", "[outv]",
        "-map", "0:a?",
        *get_gpu_encoder_args(hardware=hardware, quality=quality),
        "-c:a", "aac",
        "-b:a", "192k",
        "-force_key_frames", f"expr:gte(t,n_forced*{clip_length})",
        "-f", "segment",
        "-segment_time", str(clip_length),
        "-segment_start_number", "1",
        "-reset_timestamps", "1",
        output_pattern
    ])

    print("Running GPU-accelerated FFmpeg render with animated progress bar...")
    subprocess.run(command, check=True)
    print(f"Done! Clips saved to '{output_folder}'.")

if __name__ == "__main__":
    ffmpeg_gpu_split_with_progress_bar(
        input_video_path="input_video.mp4",
        output_folder="shorts_with_progressbar",
        clip_length=30,
        keep_remainder=False,
        hardware="auto",
        bar_height=12,              # Thickness of the progress bar in pixels
        bar_color="#FF0050",        # TikTok red/pink (try "#00F2FE" for cyan or "#FFD700" for gold)
        bar_position="video_bottom"  # "video_bottom", "screen_top", or "screen_bottom"
    )
```

How the Progress Bar Filter Works

- **Contrast Track (`black@0.45`):** Before drawing the colored bar, `drawbox` lays down a 45%-opacity dark track (`color=black@0.45:t=fill`) so the progress bar stays clearly visible even over bright video scenes.
- **Synced Bar Source (`[bar_src]`):** Rather than spawning an external synthetic color generator (which can cause frame-rate mismatches), the script clones the third stream from `[0:v]`, resizes it to `1080x12`, and fills it with your chosen hex color so it inherits the exact frame rate and duration of your video.
- **Changing Position:**
  - `"video_bottom"` *(default)*: Glues the progress bar to the bottom border of the 16:9 foreground video.
  - `"screen_top"`: Places the bar across the top edge of the 9:16 vertical frame.
  - `"screen_bottom"`: Places the bar across the bottom edge of the 9:16 vertical frame.

User prompt: Update the FFmpeg script to also draw a title banner that automatically increments 'Part 1', 'Part 2', 'Part 3' on each 30-second clip.

Response: To make the title banner automatically update to **Part 1**, **Part 2**, **Part 3**, etc. in a single FFmpeg pass, you can use the `drawtext` filter's expression expansion (`%{eif:...}`).

By passing the formula `%{eif:trunc((t+0.01)/30)+1:d}`, FFmpeg divides the current timestamp `t` by your 30-second clip length, rounds down to the nearest whole number, and adds `1`:

- **0s to 29.9s:** evaluates to `Part 1`
- **30s to 59.9s:** evaluates to `Part 2`
- **60s to 89.9s:** evaluates to `Part 3`

(Adding `0.01` to `t` prevents floating-point rounding glitches right on the 30.000-second cut boundary).

GPU-Accelerated FFmpeg Script (Blurred 9:16 + Progress Bar + Auto "Part X" Banner)

```
import os
import platform
import subprocess

def get_video_duration(input_video_path: str) -> float:
    """Uses ffprobe to get the exact video duration in seconds."""
    command = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format=duration",
        "-of", "default=noprint_wrappers=1:nokey=1",
        input_video_path
    ]
    result = subprocess.run(command, capture_output=True, text=True, check=True)
    return float(result.stdout.strip())

def get_default_font_arg() -> str:
    """Locates a bold system font across Windows, macOS, and Linux and escapes it for FFmpeg."""
    candidates = [
        "C:/Windows/Fonts/arialbd.ttf",                       # Windows
        "/System/Library/Fonts/Supplemental/Arial Bold.ttf",   # macOS
        "/System/Library/Fonts/Helvetica.ttc",                 # macOS fallback
        "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf" # Linux
    ]
    for path in candidates:
        if os.path.exists(path):
            # Escape colons for FFmpeg filter syntax (especially 'C\:/' on Windows)
            escaped_path = path.replace("\\", "/").replace(":", "\\:")
            return f"fontfile='{escaped_path}':"
    return ""  # Fallback to FFmpeg's default fontconfig if no path matches

def get_gpu_encoder_args(hardware: str = "auto", quality: int = 20) -> list:
    """Returns FFmpeg video encoder flags for NVIDIA, Apple Silicon, Intel, AMD, or CPU."""
    if hardware == "auto":
        if platform.system() == "Darwin":
            hardware = "apple"
        else:
            res = subprocess.run(
                ["ffmpeg", "-hide_banner", "-encoders"],
                capture_output=True, text=True
            )
            encoders_out = res.stdout
            if "h264_nvenc" in encoders_out:
                hardware = "nvidia"
            elif "h264_qsv" in encoders_out:
                hardware = "intel"
            elif "h264_amf" in encoders_out:
                hardware = "amd"
            else:
                hardware = "cpu"

    print(f"Using encoder profile: {hardware.upper()}")

    if hardware == "nvidia":
        return ["-c:v", "h264_nvenc", "-preset", "p4", "-rc", "vbr", "-cq", str(quality), "-b:v", "0"]
    elif hardware == "apple":
        apple_q = max(1, min(100, 100 - int(quality * 1.75)))
        return ["-c:v", "h264_videotoolbox", "-q:v", str(apple_q), "-allow_sw", "1"]
    elif hardware == "intel":
        return ["-c:v", "h264_qsv", "-preset", "fast", "-global_quality", str(quality)]
    elif hardware == "amd":
```

```
            return ["-c:v", "h264_amf", "-quality", "balanced", "-rc", "cqp", "-qp_i", str(quality), "-qp_p", str(qualit
        else:
            return ["-c:v", "libx264", "-preset", "fast", "-crf", str(quality)]

    def ffmpeg_gpu_shorts_generator(
        input_video_path: str,
        output_folder: str,
        clip_length: int = 30,
        keep_remainder: bool = False,
        hardware: str = "auto",
        quality: int = 20,
        bar_height: int = 12,
        bar_color: str = "#FF0050",
        bar_position: str = "video_bottom",
        title_prefix: str = "",            # e.g., "Highlights - " -> renders "Highlights - Part 1"
        banner_y: int = 440,               # Vertical position of the banner (above the 16:9 video)
        font_size: int = 64
    ):
        os.makedirs(output_folder, exist_ok=True)
        output_pattern = os.path.join(output_folder, "%03d.mp4")

        # 1. Animated progress bar X-coordinate expression (resets every `clip_length` seconds)
        progress_x_expr = f"-W+W*(t/{clip_length}-floor(t/{clip_length}))"

        # 2. Auto-incrementing "Part 1, Part 2..." expression
        safe_prefix = title_prefix.replace(":", "\\:").replace("'", "\\'")
        part_text_expr = f"{safe_prefix}Part %{{eif:trunc((t+0.01)/{clip_length})+1:d}}"
        font_arg = get_default_font_arg()

        drawtext_filter = (
            f"drawtext={font_arg}"
            f"text='{part_text_expr}':"
            f"fontcolor=white:fontsize={font_size}:"
            f"box=1:boxcolor=black@0.75:boxborderw=24:"
            f"x=(w-text_w)/2:y={banner_y}"
        )

        # 3. Assemble the complete filter_complex pipeline
        if bar_position == "video_bottom":
            filter_complex = (
                f"[0:v]split=3[bg][fg][bar_src];"
                f"[bg]scale=270:480:force_original_aspect_ratio=increase,"
                f"crop=270:480,gblur=sigma=10,scale=1080:1920:flags=bicubic[bg_blurred];"
                f"[fg]scale=1080:-2,"
                f"drawbox=x=0:y=ih-{bar_height}:w=iw:h={bar_height}:color=black@0.45:t=fill[fg_track];"
                f"[bar_src]scale=1080:{bar_height},drawbox=color={bar_color}:t=fill[bar];"
                f"[fg_track][bar]overlay=x='{progress_x_expr}':y=H-h[fg_with_bar];"
                f"[bg_blurred][fg_with_bar]overlay=(W-w)/2:(H-h)/2,"
                f"{drawtext_filter},format=yuv420p[outv]"
            )
        else:
            y_pos = "0" if bar_position == "screen_top" else f"1920-{bar_height}"
            filter_complex = (
                f"[0:v]split=3[bg][fg][bar_src];"
                f"[bg]scale=270:480:force_original_aspect_ratio=increase,"
                f"crop=270:480,gblur=sigma=10,scale=1080:1920:flags=bicubic[bg_blurred];"
                f"[fg]scale=1080:-2[fg_scaled];"
                f"[bg_blurred][fg_scaled]overlay=(W-w)/2:(H-h)/2,"
                f"drawbox=x=0:y={y_pos}:w=1080:h={bar_height}:color=black@0.45:t=fill[canvas_track];"
                f"[bar_src]scale=1080:{bar_height},drawbox=color={bar_color}:t=fill[bar];"
                f"[canvas_track][bar]overlay=x='{progress_x_expr}':y={y_pos},"
                f"{drawtext_filter},format=yuv420p[outv]"
            )

        command = [
            "ffmpeg",
            "-y",
            "-hwaccel", "auto",
            "-i", input_video_path,
        ]

        if not keep_remainder:
            total_duration = get_video_duration(input_video_path)
            usable_duration = (int(total_duration) // clip_length) * clip_length
            if usable_duration == 0:
                raise ValueError(f"Video is shorter than {clip_length} seconds.")
            command.extend(["-t", str(usable_duration)])

        command.extend([
```

```
            "-filter_complex", filter_complex,
            "-map", "[outv]",
            "-map", "0:a?",
            *get_gpu_encoder_args(hardware=hardware, quality=quality),
            "-c:a", "aac",
            "-b:a", "192k",
            "-force_key_frames", f"expr:gte(t,n_forced*{clip_length})",
            "-f", "segment",
            "-segment_time", str(clip_length),
            "-segment_start_number", "1",
            "-reset_timestamps", "1",
            output_pattern
        ])

        print("Running GPU-accelerated FFmpeg render with 'Part X' banner and progress bar...")
        subprocess.run(command, check=True)
        print(f"Done! Clips saved to '{output_folder}'.")

if __name__ == "__main__":
    ffmpeg_gpu_shorts_generator(
        input_video_path="input_video.mp4",
        output_folder="shorts_complete",
        clip_length=30,
        keep_remainder=False,
        hardware="auto",
        bar_height=12,
        bar_color="#FF0050",
        bar_position="video_bottom",
        title_prefix="",  # Set to e.g. "Best Moments - " to display "Best Moments - Part 1"
        banner_y=440      # Sits cleanly in the blurred space above the centered 16:9 video
    )
```

How the Banner Styling & Positioning Works

- **Smart Placement (`banner_y=440`):** On a `1080x1920` canvas, your centered 16:9 video occupies the vertical range from `y=656` to `y=1264`. Placing the banner at `y=440` centers it inside the upper blurred region—high enough to sit clear of the video, and low enough to avoid the top UI icons on TikTok, Reels, and Shorts.
- **Auto-Sizing Background Pill (`box=1:boxcolor=black@0.75:boxborderw=24`):** Instead of drawing a static full-width rectangle, `drawtext`'s built-in `box=1` creates a dark, semi-transparent background badge with 24 pixels of padding that automatically widens when `"Part 9"` becomes `"Part 10"`.
- **Custom Title Prefix (`title_prefix`):** Leave `title_prefix=""` to display `"Part 1"`, `"Part 2"`, or pass a prefix like `title_prefix="Podcast Ep. 4 - "` to render `"Podcast Ep. 4 - Part 1"`.

User prompt: give me instructions to setup this up on windows 11 and windows 10

Response: Setting up the GPU-accelerated FFmpeg Python script on **Windows 11** and **Windows 10** only takes a few minutes. Because the final script uses Python's built-in `subprocess` module to talk directly to FFmpeg, you do not even need to `pip install` any third-party Python packages.

1 **Open PowerShell as Administrator**

Required to install system-wide tools and update your PATH automatically

Press **Win + X** on your keyboard and select:

- **Windows 11: Terminal (Admin)**
- **Windows 10: Windows PowerShell (Admin)** or **Command Prompt (Admin)**

Click **Yes** on the User Account Control (UAC) prompt.

2 **Install Python and FFmpeg**

Uses Winget (built into Windows 11 and modern Windows 10)

Run the following two commands one at a time in your Administrator terminal. `winget` will download, install, and automatically add both tools to your Windows System `PATH`:

```
winget install -e --id Python.Python.3.12
winget install -e --id Gyan.FFmpeg
```

*If prompted to agree to source terms, type `Y` and press **Enter**.*

Fallback for older Windows 10 builds without `winget`:

1. **Python:** Download the installer from python.org. On the very first installer screen, **check the box that says "Add python.exe to PATH"** before clicking *Install Now*.

2. **FFmpeg:** Download `ffmpeg-release-essentials.zip` from gyan.dev/ffmpeg/builds, extract it to `C:\ffmpeg`, and add `C:\ffmpeg\bin` to your Windows Environment Variables under **Path**.

3 **Restart Terminal and Verify Installations**

You must close and reopen your terminal for PATH changes to take effect

Close the Administrator terminal window completely, open a new regular Command Prompt or PowerShell window, and run:

```
python --version
ffmpeg -version
ffprobe -version
```

If all three commands print version numbers instead of *"not recognized"* errors, your environment is ready.

4 **Create Your Project Folder and Script File**

1. Open **File Explorer** and create a new folder (for example, `C:\ShortsGenerator`).
2. Copy the video you want to split into `C:\ShortsGenerator` and rename it to **`input_video.mp4`**.
3. Open **Notepad** (or VS Code), paste the complete Python script from above, and save it inside `C:\ShortsGenerator` as **`make_shorts.py`**.
    - *Windows Notepad tip:* In the **Save As** dialog, change **Save as type** from `Text Documents (*.txt)` to **`All Files (*.*)`** so Windows does not secretly name the file `make_shorts.py.txt`.

5 **Run the Generator**

Open a terminal directly inside your `C:\ShortsGenerator` folder:

- **Windows 11 & 10 shortcut:** Click the address bar at the top of File Explorer, type **`cmd`**, and press **Enter**.

In the window that opens, run:

```
python make_shorts.py
```

FFmpeg will automatically detect your GPU (`NVIDIA`, `INTEL`, or `AMD`), render the blurred 9:16 background, animate the progress bar, increment the **Part 1, Part 2...** banner using Windows' built-in Arial Bold font (`C:\Windows\Fonts\arialbd.ttf`), and save your numbered clips (`001.mp4`, `002.mp4`, etc.) inside `C:\ShortsGenerator\shorts_complete`.

Common Windows Troubleshooting

Issue / Error Message	Cause	Fix on Windows 10 & 11
`'ffmpeg' is not recognized` or `FileNotFoundError: [WinError 2]`	Terminal was not restarted after installing FFmpeg, or FFmpeg is missing from `PATH`.	Close **all** open Command Prompt / PowerShell / VS Code windows and open a fresh one. If it persists, press **Win + R**, run `rundll32.exe sysdm.cpl,EditEnvironmentVariables`, edit **Path**, and ensure the folder containing `ffmpeg.exe` is listed.
`Python was not found; run without arguments to install from the Microsoft Store`	Windows App Execution Aliases are intercepting the `python` command.	Open **Settings** -> **Apps** -> **Advanced app settings** (or *Apps & features* on Win 10) -> **App execution aliases**, and toggle **Off** both `python.exe` and `python3.exe`.
`Driver does not support the required nvenc API version`	Outdated NVIDIA graphics driver.	Update your GPU driver via the **NVIDIA App** / **GeForce Experience**, or change `hardware="auto"` to `hardware="cpu"` at the bottom of `make_shorts.py`.

User prompt: Make me a Windows .bat batch file so I can just drag and drop any video file onto it to generate the 30-second Shorts clips automatically.

Response: To make drag-and-drop work on Windows, you need two files in the same folder:

1. **`Drop_Video_Here.bat`** — receives the file(s) you drag onto it and passes their file paths to Python.
2. **An updated `make_shorts.py`** — reads the dragged file path using `sys.argv` and automatically creates a matching output folder right next to your original video (for example, dragging `podcast.mp4` creates a `podcast_shorts` folder).

1. Create `Drop_Video_Here.bat`

Open **Notepad**, paste the batch script below, and save it in `C:\ShortsGenerator` as **`Drop_Video_Here.bat`** (make sure **Save as type** is set to **All Files (`*.*`)**):

```
@echo off
title Shorts Clip Generator

:: Check if at least one file was dragged onto the batch file
if "%~1"=="" (
```

```
    echo ========================================================
    echo  Drag and drop one or more video files onto this icon!
    echo ========================================================
    pause
    exit /b
)

:: Loop through all dragged files (supports dragging multiple videos at once)
for %%F in (%*) do (
    echo.
    echo ------------------------------------------------------
    echo Processing: "%%~nxF"
    echo ------------------------------------------------------
    python "%~dp0make_shorts.py" "%%~fF"
)

echo.
echo ========================================================
echo  All videos finished processing!
echo ========================================================
pause
```

- **Works from anywhere on your PC:** `%~dp0` tells Windows to always look for `make_shorts.py` inside the folder where the `.bat` file lives, even if you drag a video from your Desktop or an external drive.
- **Multi-video support:** You can highlight several videos at once and drag them all onto `Drop_Video_Here.bat`—it will queue and split them one after another.

2. Update `make_shorts.py` to Accept Dragged Files

Replace the contents of `make_shorts.py` (in the same folder as your `.bat` file) with this updated version. The `if __name__ == "__main__":` block at the bottom now detects dragged files automatically and creates a dedicated output folder for each video so clips never overwrite each other:

```
import os
import sys
import platform
import subprocess

def get_video_duration(input_video_path: str) -> float:
    """Uses ffprobe to get the exact video duration in seconds."""
    command = [
        "ffprobe",
        "-v", "error",
        "-show_entries", "format=duration",
        "-of", "default=noprint_wrappers=1:nokey=1",
        input_video_path
    ]
    result = subprocess.run(command, capture_output=True, text=True, check=True)
    return float(result.stdout.strip())

def get_default_font_arg() -> str:
    """Locates a bold system font across Windows, macOS, and Linux and escapes it for FFmpeg."""
    candidates = [
        "C:/Windows/Fonts/arialbd.ttf",                        # Windows
        "/System/Library/Fonts/Supplemental/Arial Bold.ttf",   # macOS
        "/System/Library/Fonts/Helvetica.ttc",                 # macOS fallback
        "/usr/share/fonts/truetype/dejavu/DejaVuSans-Bold.ttf" # Linux
    ]
    for path in candidates:
        if os.path.exists(path):
            escaped_path = path.replace("\\", "/").replace(":", "\\:")
            return f"fontfile='{escaped_path}':"
    return ""

def get_gpu_encoder_args(hardware: str = "auto", quality: int = 20) -> list:
    """Returns FFmpeg video encoder flags for NVIDIA, Apple Silicon, Intel, AMD, or CPU."""
    if hardware == "auto":
        if platform.system() == "Darwin":
            hardware = "apple"
        else:
            res = subprocess.run(
                ["ffmpeg", "-hide_banner", "-encoders"],
                capture_output=True, text=True
            )
            encoders_out = res.stdout
            if "h264_nvenc" in encoders_out:
                hardware = "nvidia"
            elif "h264_qsv" in encoders_out:
                hardware = "intel"
```

```
            elif "h264_amf" in encoders_out:
                hardware = "amd"
            else:
                hardware = "cpu"

        print(f"Using encoder profile: {hardware.upper()}")

        if hardware == "nvidia":
            return ["-c:v", "h264_nvenc", "-preset", "p4", "-rc", "vbr", "-cq", str(quality), "-b:v", "0"]
        elif hardware == "apple":
            apple_q = max(1, min(100, 100 - int(quality * 1.75)))
            return ["-c:v", "h264_videotoolbox", "-q:v", str(apple_q), "-allow_sw", "1"]
        elif hardware == "intel":
            return ["-c:v", "h264_qsv", "-preset", "fast", "-global_quality", str(quality)]
        elif hardware == "amd":
            return ["-c:v", "h264_amf", "-quality", "balanced", "-rc", "cqp", "-qp_i", str(quality), "-qp_p", str(qualit
        else:
            return ["-c:v", "libx264", "-preset", "fast", "-crf", str(quality)]

    def ffmpeg_gpu_shorts_generator(
        input_video_path: str,
        output_folder: str,
        clip_length: int = 30,
        keep_remainder: bool = False,
        hardware: str = "auto",
        quality: int = 20,
        bar_height: int = 12,
        bar_color: str = "#FF0050",
        bar_position: str = "video_bottom",
        title_prefix: str = "",
        banner_y: int = 440,
        font_size: int = 64
    ):
        os.makedirs(output_folder, exist_ok=True)
        output_pattern = os.path.join(output_folder, "%03d.mp4")

        progress_x_expr = f"-W+W*(t/{clip_length}-floor(t/{clip_length}))"

        safe_prefix = title_prefix.replace(":", "\\:").replace("'", "\\'")
        part_text_expr = f"{safe_prefix}Part %{{eif:trunc((t+0.01)/{clip_length})+1:d}}"
        font_arg = get_default_font_arg()

        drawtext_filter = (
            f"drawtext={font_arg}"
            f"text='{part_text_expr}':"
            f"fontcolor=white:fontsize={font_size}:"
            f"box=1:boxcolor=black@0.75:boxborderw=24:"
            f"x=(w-text_w)/2:y={banner_y}"
        )

        if bar_position == "video_bottom":
            filter_complex = (
                f"[0:v]split=3[bg][fg][bar_src];"
                f"[bg]scale=270:480:force_original_aspect_ratio=increase,"
                f"crop=270:480,gblur=sigma=10,scale=1080:1920:flags=bicubic[bg_blurred];"
                f"[fg]scale=1080:-2,"
                f"drawbox=x=0:y=ih-{bar_height}:w=iw:h={bar_height}:color=black@0.45:t=fill[fg_track];"
                f"[bar_src]scale=1080:{bar_height},drawbox=color={bar_color}:t=fill[bar];"
                f"[fg_track][bar]overlay=x='{progress_x_expr}':y=H-h[fg_with_bar];"
                f"[bg_blurred][fg_with_bar]overlay=(W-w)/2:(H-h)/2,"
                f"{drawtext_filter},format=yuv420p[outv]"
            )
        else:
            y_pos = "0" if bar_position == "screen_top" else f"1920-{bar_height}"
            filter_complex = (
                f"[0:v]split=3[bg][fg][bar_src];"
                f"[bg]scale=270:480:force_original_aspect_ratio=increase,"
                f"crop=270:480,gblur=sigma=10,scale=1080:1920:flags=bicubic[bg_blurred];"
                f"[fg]scale=1080:-2[fg_scaled];"
                f"[bg_blurred][fg_scaled]overlay=(W-w)/2:(H-h)/2,"
                f"drawbox=x=0:y={y_pos}:w=1080:h={bar_height}:color=black@0.45:t=fill[canvas_track];"
                f"[bar_src]scale=1080:{bar_height},drawbox=color={bar_color}:t=fill[bar];"
                f"[canvas_track][bar]overlay=x='{progress_x_expr}':y={y_pos},"
                f"{drawtext_filter},format=yuv420p[outv]"
            )

        command = [
            "ffmpeg",
            "-y",
```

```
        "-hwaccel", "auto",
        "-i", input_video_path,
    ]

    if not keep_remainder:
        total_duration = get_video_duration(input_video_path)
        usable_duration = (int(total_duration) // clip_length) * clip_length
        if usable_duration == 0:
            raise ValueError(f"Video '{input_video_path}' is shorter than {clip_length} seconds.")
        command.extend(["-t", str(usable_duration)])

    command.extend([
        "-filter_complex", filter_complex,
        "-map", "[outv]",
        "-map", "0:a?",
        *get_gpu_encoder_args(hardware=hardware, quality=quality),
        "-c:a", "aac",
        "-b:a", "192k",
        "-force_key_frames", f"expr:gte(t,n_forced*{clip_length})",
        "-f", "segment",
        "-segment_time", str(clip_length),
        "-segment_start_number", "1",
        "-reset_timestamps", "1",
        output_pattern
    ])

    print(f"Rendering clips into: {output_folder}")
    subprocess.run(command, check=True)
    print(f"Done! Clips saved to '{output_folder}'.")

if __name__ == "__main__":
    # Get dragged video path from command line, or default to "input_video.mp4"
    input_path = sys.argv[1] if len(sys.argv) > 1 else "input_video.mp4"

    if not os.path.exists(input_path):
        print(f"Error: File not found -> {input_path}")
        sys.exit(1)

    # Create an output folder right next to the input video named "<videoname>_shorts"
    video_dir = os.path.dirname(os.path.abspath(input_path))
    video_stem = os.path.splitext(os.path.basename(input_path))[0]
    auto_output_folder = os.path.join(video_dir, f"{video_stem}_shorts")

    ffmpeg_gpu_shorts_generator(
        input_video_path=input_path,
        output_folder=auto_output_folder,
        clip_length=30,
        keep_remainder=False,
        hardware="auto",
        bar_height=12,
        bar_color="#FF0050",
        bar_position="video_bottom",
        title_prefix="",
        banner_y=440
    )
```

Pro Tip: Put a Drag-and-Drop Shortcut on Your Desktop

You don't need to keep the `C:\ShortsGenerator` folder open to use this:

1. Right-click **`Drop_Video_Here.bat`** inside `C:\ShortsGenerator`.
2. Select **Show more options** (on Windows 11) -> **Send to** -> **Desktop (create shortcut)**.
3. Now you can drag and drop any `.mp4`, `.mkv`, or `.mov` file directly onto the Desktop shortcut, and a new `<videoname>_shorts` folder containing `001.mp4`, `002.mp4`, etc. will appear right next to your original video.

User prompt: make a readme.me file with: - How this works? - How to setup

Response: Save the content below as **`README.md`** inside your `C:\ShortsGenerator` folder (or your GitHub repository):

```
# 🎬 Automated 9:16 Shorts, Reels & TikTok Clip Generator

A zero-dependency, GPU-accelerated Python & FFmpeg tool that automatically converts widescreen (16:9) videos into nu

---

## ⚙️ How This Works
```

Instead of decoding and re-encoding video frames slowly in Python, this tool builds a **single-pass FFmpeg `filter_c

1. **Single-Pass Stream Cloning (`split=3`):**
   FFmpeg reads the input video once and splits the video stream in memory into three synchronized paths: the backgr
2. **Fast 4x Downscale-Blur Background (`1080x1920`):**
   Applying a heavy Gaussian blur to a full 1080p frame on the CPU creates a major bottleneck. The pipeline scales t
3. **Centered 16:9 Foreground:**
   Your original widescreen video is scaled to `1080` pixels wide (preserving its aspect ratio) and overlaid dead-ce
4. **Animated Resetting Progress Bar:**
   Because FFmpeg's `drawbox` cannot animate width frame-by-frame, the script generates a solid color bar (`#FF0050`
   ```text
   x = -W + W * (t/30 - floor(t/30))
   ```
   This smoothly fills the progress bar from `0%` to `100%` over 30 seconds and automatically resets to `0%` at `30s
5. **Auto-Incrementing "Part X" Banner:**
   Using FFmpeg's dynamic expression evaluation (`%{eif:trunc((t+0.01)/30)+1:d}`), the `drawtext` filter calculates
6. **Hardware GPU Encoding & Frame-Exact Splitting:**
   * **Auto GPU Detection:** Queries your system and selects hardware encoding automatically—supporting **NVIDIA NVE
   * **Exact Cuts:** Forces keyframes at exact 30-second intervals (`-force_key_frames`) and uses FFmpeg's `segment`

🚀 How to Set Up (Windows 10 & Windows 11)

Step 1: Install Python and FFmpeg
1. Press **Win + X** and select **Terminal (Admin)** (Windows 11) or **Windows PowerShell (Admin)** (Windows 10).
2. Run the following commands to install Python and FFmpeg and automatically add them to your system `PATH`:
   ```powershell
   winget install -e --id Python.Python.3.12
   winget install -e --id Gyan.FFmpeg
   ```
   *(If you are on an older Windows 10 build without `winget`, install Python from [python.org](https://www.python.o
3. **Close and reopen** your terminal window, then verify both tools are installed:
   ```powershell
   python --version
   ffmpeg -version
   ffprobe -version
   ```

Step 2: Set Up the Project Folder
Create a folder (e.g., `C:\ShortsGenerator`) containing these two files:

```text
C:\ShortsGenerator\
├── make_shorts.py        # Main Python + FFmpeg generator script
├── Drop_Video_Here.bat   # Drag-and-drop Windows batch launcher
└── README.md             # Documentation
```

*(Note: When saving `make_shorts.py` and `Drop_Video_Here.bat` in Notepad, set **Save as type** to **All Files (`*.*

Step 3: Run the Generator

Option A: Drag and Drop (Recommended)
1. Drag one or more video files (`.mp4`, `.mkv`, `.mov`) directly onto **`Drop_Video_Here.bat`** (or create a shortc
2. A folder named `<videoname>_shorts` will automatically be created right next to your source video containing `001

Option B: Command Line
Run the script from Command Prompt or PowerShell by passing the path to any video file:
```cmd
python make_shorts.py "C:\path\to\my_video.mp4"
```

🎨 Customization Options

You can customize the look and behavior at the bottom of `make_shorts.py` inside the `ffmpeg_gpu_shorts_generator(..

Parameter	Default	Description
`clip_length`	`30`	Duration of each output clip in seconds.
`keep_remainder`	`False`	`False` drops any leftover tail shorter than `clip_length`; `True` exports the final
`hardware`	`"auto"`	`"auto"`, `"nvidia"`, `"apple"`, `"intel"`, `"amd"`, or `"cpu"`.
`quality`	`20`	Encoding quality target (`18`–`24` recommended; lower number = higher visual quality).
`bar_height`	`12`	Thickness of the animated progress bar in pixels.
`bar_color`	`"#FF0050"`	Hex color or name for the progress bar (e.g., `"#FF0050"`, `"#00F2FE"`, `"white"`).
`bar_position`	`"video_bottom"`	`"video_bottom"` (flush with 16:9 clip), `"screen_top"`, or `"screen_bottom"`.

```
| `title_prefix` | `""` | Optional text before the part number (e.g., `"Highlights - "` renders `"Highlights - Part
| `banner_y` | `440` | Vertical Y-coordinate of the title banner on the `1080x1920` canvas. |
| `font_size` | `64` | Font size of the `"Part X"` banner text. |

---

## 🛠️ Troubleshooting

* **`'ffmpeg' is not recognized as an internal or external command`:**
  Close all open terminal windows and open a new one so Windows reloads your `PATH`.
* **`Python was not found; run without arguments to install from the Microsoft Store`:**
  Go to **Windows Settings -> Apps -> Advanced app settings -> App execution aliases** and toggle **Off** `python.ex
* **`Driver does not support the required nvenc API version`:**
  Update your NVIDIA GPU drivers via the NVIDIA App, or set `hardware="cpu"` in `make_shorts.py`.
```

User prompt: add the dependencys and a note that this is "vide coded"

Response: Save the updated file below as **`README.md`**. It now includes a dedicated **Dependencies & Prerequisites** section (covering both the zero-`pip` FFmpeg version and the optional MoviePy version from earlier) along with a **Vibe Coded** badge and disclaimer at the top:

```
# 🎬 Automated 9:16 Shorts, Reels & TikTok Clip Generator

> ✨ **Note: This project was 100% "Vibe Coded"** *(vide coded)* — built iteratively through conversational AI promp

A GPU-accelerated Python & FFmpeg tool that automatically converts widescreen (16:9) videos into numbered, 30-second

---

## 📦 Dependencies & Prerequisites

### 1. System Dependencies (Required)
* **Python 3.8+** — Uses only Python's built-in standard library modules (`os`, `sys`, `platform`, `subprocess`, `ma
* **FFmpeg & FFprobe** — Must be installed and available in your system `PATH` (includes `libx264`, `drawtext`, `gbl
* **System Font** — Uses standard pre-installed OS fonts for the `"Part X"` banner (`arialbd.ttf` on Windows, `Arial

### 2. Optional Hardware Acceleration
* **NVIDIA GPU:** NVIDIA Driver `450.0+` (for `h264_nvenc`)
* **Apple Silicon:** M1 / M2 / M3 / M4 Mac (for `h264_videotoolbox`)
* **Intel / AMD:** Intel QuickSync (`h264_qsv`) or AMD Radeon AMF (`h264_amf`)

### 3. Optional Python Packages *(Only if using the legacy MoviePy script)*
If you use the pure-Python MoviePy version instead of the fast FFmpeg `subprocess` script:
```bash
pip install moviepy numpy Pillow
```

⚙️ How This Works

Instead of decoding and re-encoding video frames slowly in Python, this tool builds a **single-pass FFmpeg `filter_c

1. **Single-Pass Stream Cloning (`split=3`):**
   FFmpeg reads the input video once and splits the video stream in memory into three synchronized paths: the backgr
2. **Fast 4x Downscale-Blur Background (`1080x1920`):**
   Applying a heavy Gaussian blur to a full 1080p frame on the CPU creates a major bottleneck. The pipeline scales t
3. **Centered 16:9 Foreground:**
   Your original widescreen video is scaled to `1080` pixels wide (preserving its aspect ratio) and overlaid dead-ce
4. **Animated Resetting Progress Bar:**
   Because FFmpeg's `drawbox` cannot animate width frame-by-frame, the script generates a solid color bar (`#FF0050`
   ```text
   x = -W + W * (t/30 - floor(t/30))
   ```
   This smoothly fills the progress bar from `0%` to `100%` over 30 seconds and automatically resets to `0%` at `30s
5. **Auto-Incrementing "Part X" Banner:**
   Using FFmpeg's dynamic expression evaluation (`%{eif:trunc((t+0.01)/30)+1:d}`), the `drawtext` filter calculates
6. **Hardware GPU Encoding & Frame-Exact Splitting:**
   * **Auto GPU Detection:** Queries your system and selects hardware encoding automatically—supporting **NVIDIA NVE
   * **Exact Cuts:** Forces keyframes at exact 30-second intervals (`-force_key_frames`) and uses FFmpeg's `segment`

🚀 How to Set Up (Windows 10 & Windows 11)

Step 1: Install Python and FFmpeg
1. Press **Win + X** and select **Terminal (Admin)** (Windows 11) or **Windows PowerShell (Admin)** (Windows 10).
2. Run the following commands to install Python and FFmpeg and automatically add them to your system `PATH`:
```

```
   ```powershell
   winget install -e --id Python.Python.3.12
   winget install -e --id Gyan.FFmpeg
   ```
   *(If you are on an older Windows 10 build without `winget`, install Python from [python.org](https://www.python.o
3. **Close and reopen** your terminal window, then verify both tools are installed:
   ```powershell
   python --version
   ffmpeg -version
   ffprobe -version
   ```

Step 2: Set Up the Project Folder
Create a folder (e.g., `C:\ShortsGenerator`) containing these files:

```text
C:\ShortsGenerator\
├── make_shorts.py        # Main Python + FFmpeg generator script
├── Drop_Video_Here.bat   # Drag-and-drop Windows batch launcher
└── README.md             # Documentation
```

*(Note: When saving `make_shorts.py` and `Drop_Video_Here.bat` in Notepad, set **Save as type** to **All Files (`*.*

Step 3: Run the Generator

Option A: Drag and Drop (Recommended)
1. Drag one or more video files (`.mp4`, `.mkv`, `.mov`) directly onto **`Drop_Video_Here.bat`** (or create a shortc
2. A folder named `<videoname>_shorts` will automatically be created right next to your source video containing `001

Option B: Command Line
Run the script from Command Prompt or PowerShell by passing the path to any video file:
```cmd
python make_shorts.py "C:\path\to\my_video.mp4"
```

🎨 Customization Options

You can customize the look and behavior at the bottom of `make_shorts.py` inside the `ffmpeg_gpu_shorts_generator(..

Parameter	Default	Description
`clip_length`	`30`	Duration of each output clip in seconds.
`keep_remainder`	`False`	`False` drops any leftover tail shorter than `clip_length`; `True` exports the final
`hardware`	`"auto"`	`"auto"`, `"nvidia"`, `"apple"`, `"intel"`, `"amd"`, or `"cpu"`.
`quality`	`20`	Encoding quality target (`18`–`24` recommended; lower number = higher visual quality).
`bar_height`	`12`	Thickness of the animated progress bar in pixels.
`bar_color`	`"#FF0050"`	Hex color or name for the progress bar (e.g., `"#FF0050"`, `"#00F2FE"`, `"white"`).
`bar_position`	`"video_bottom"`	`"video_bottom"` (flush with 16:9 clip), `"screen_top"`, or `"screen_bottom"`.
`title_prefix`	`""`	Optional text before the part number (e.g., `"Highlights - "` renders `"Highlights - Part
`banner_y`	`440`	Vertical Y-coordinate of the title banner on the `1080x1920` canvas.
`font_size`	`64`	Font size of the `"Part X"` banner text.

🛠️ Troubleshooting

* **`'ffmpeg' is not recognized as an internal or external command`:**
  Close all open terminal windows and open a new one so Windows reloads your `PATH`.
* **`Python was not found; run without arguments to install from the Microsoft Store`:**
  Go to **Windows Settings -> Apps -> Advanced app settings -> App execution aliases** and toggle **Off** `python.ex
* **`Driver does not support the required nvenc API version`:**
  Update your NVIDIA GPU drivers via the NVIDIA App, or set `hardware="cpu"` in `make_shorts.py`.
```

User prompt: Create a setup.ps1 PowerShell script that automatically checks for Python and FFmpeg, installs them via winget if missing, and creates a Desktop shortcut to Drop_Video_Here.bat.

Response: Save the script below as **`setup.ps1`** inside your `C:\ShortsGenerator` folder alongside your other files.

This automated PowerShell script handles everything: it checks if Python and FFmpeg are installed, installs them via `winget` if missing, refreshes your environment path, and creates a convenient **Shorts Generator** shortcut right on your Windows Desktop.

`setup.ps1` Script Code

```
# ============================================================================
# Automated Setup Script for Shorts Generator (Windows 10 & 11)
# ============================================================================

# 1. Ensure script is running with Administrator privileges
if (-not ([Security.Principal.WindowsPrincipal][Security.Principal.WindowsIdentity]::GetCurrent()).IsInRole([Securit
    Write-Host "[!] Requesting Administrator privileges..." -ForegroundColor Yellow
    Start-Process powershell.exe "-NoProfile -ExecutionPolicy Bypass -File `"$PSCommandPath`"" -Verb RunAs
    Exit
}

# Bind execution path to where this script lives
Set-Location $PSScriptRoot
Write-Host "============================================================" -ForegroundColor Cyan
Write-Host " Setting up Shorts Generator Environment..." -ForegroundColor Cyan
Write-Host "============================================================" -ForegroundColor Cyan

# 2. Check and Install Python
$pythonInstalled =$false
try {
    $null = Get-Command python -ErrorAction Stop
    $pythonInstalled =$true
} catch {
    # Check common winget install path fallback if PATH hasn't updated yet
    if (Test-Path "$env:LOCALAPPDATA\Programs\Python\Python312\python.exe") {
        $pythonInstalled =$true
    }
}

if ($pythonInstalled) {
    Write-Host "[+] Python is already installed." -ForegroundColor Green
} else {
    Write-Host "[*] Python not found. Installing via winget..." -ForegroundColor Yellow
    winget install -e --id Python.Python.3.12 --accept-source-agreements --accept-package-agreements
}

# 3. Check and Install FFmpeg
$ffmpegInstalled =$false
try {
    $null = Get-Command ffmpeg -ErrorAction Stop
    $ffmpegInstalled =$true
} catch {
    if (Test-Path "C:\ffmpeg\bin\ffmpeg.exe") {
        $ffmpegInstalled =$true
    }
}

if ($ffmpegInstalled) {
    Write-Host "[+] FFmpeg is already installed." -ForegroundColor Green
} else {
    Write-Host "[*] FFmpeg not found. Installing via winget..." -ForegroundColor Yellow
    winget install -e --id Gyan.FFmpeg --accept-source-agreements --accept-package-agreements
}

# 4. Refresh Environment PATH for the current session
$env:Path = [System.Environment]::GetEnvironmentVariable("Path","Machine") + ";" + [System.Environment]::GetEnvironm

# 5. Create Desktop Shortcut to Drop_Video_Here.bat
$batPath = Join-Path$PSScriptRoot "Drop_Video_Here.bat"
if (Test-Path $batPath) {$desktopPath = [Environment]::GetFolderPath("Desktop")
    $shortcutPath = Join-Path$desktopPath "Shorts Generator.lnk"

    $WScriptShell = New-Object -ComObject WScript.Shell
    $Shortcut =$WScriptShell.CreateShortcut($shortcutPath)$Shortcut.TargetPath = $batPath$Shortcut.WorkingDirectory
    $Shortcut.Save()

    Write-Host "[+] Desktop shortcut created successfully!" -ForegroundColor Green
} else {
    Write-Host "[!] Warning: 'Drop_Video_Here.bat' not found in $PSScriptRoot. Shortcut skipped." -ForegroundColor Y
}

Write-Host ""
Write-Host "============================================================" -ForegroundColor Cyan
Write-Host " Setup complete! You can now drag videos onto your Desktop shortcut." -ForegroundColor Cyan
Write-Host "============================================================" -ForegroundColor Cyan
Read-Host -Prompt "Press Enter to exit"
```

How to Run the Setup Script

1. Make sure `make_shorts.py`, `Drop_Video_Here.bat`, and `setup.ps1` are all saved in the same folder (e.g., `C:\ShortsGenerator`).
2. Right-click **`setup.ps1`** and select **Run with PowerShell**.
3. Click **Yes** on the Administrator permission popup.
4. The script will automatically verify or install Python and FFmpeg, configure your system paths, and drop a **Shorts Generator** icon right onto your Windows Desktop!
